

Tarea 1: Redes Neuronales Convolucionales

IIC3697 – Aprendizaje Profundo

Jorge Ignacio Dimter Arce

Sección 1: CNNs y Conexiones Residuales

Actividad 1

¿Cuál es la función de las conexiones residuales dentro de un bloque ResNet?

Las conexiones residuales permiten que la entrada de un bloque se sume directamente a la salida de las capas, antes de la activación final. En lugar de que la red aprenda la transformación completa $H(x)$, aprende el residuo $F(x) = H(x) - x$, lo que en la práctica es más sencillo de optimizar. Esto facilita el flujo del gradiente durante el backpropagation, ya que existe un camino directo que evita que el gradiente se desvanezca al atravesar muchas capas y también permite combinar representaciones de distinto nivel de abstracción, dado que la información de capas anteriores llega sin ser transformada directamente a capas más profundas.

¿En qué casos es necesario modificar la conexión residual (shortcut) en términos de dimensiones?

La suma residual solo es posible si el tensor de entrada tiene exactamente las mismas dimensiones espaciales y de canales que el tensor de salida del bloque. Cuando esto no ocurre, el shortcut debe adaptarse mediante una convolución 1×1 seguida de BatchNorm. Esto sucede en dos situaciones: cuando el stride es mayor a 1 (lo que reduce la resolución espacial) o cuando el número de canales cambia entre entrada y salida. En la implementación de ResNet18, esto ocurre al inicio de los stages 2, 3 y 4, donde los canales se duplican y se aplica stride=2.

Actividad 2

¿Qué cambio introduce la eliminación de las skip connections en la forma en que la red aprende las representaciones?

Al quitar las skip connections, básicamente obligamos a la red a aprender una transformación completa en cada bloque, en lugar de solo aprender el residuo.º lo que falta. Sin el camino directo, la información se ve forzada a pasar sí o sí por todas las capas convolucionales, lo

que hace que las capas más profundas pierdan el acceso fácil a las características básicas (de bajo nivel) que se extrajeron al principio.

¿Qué dificultades podrían surgir al entrenar redes profundas sin conexiones residuales?

El principal problema es el desvanecimiento del gradiente: al propagarse hacia atrás a través de muchas capas, el gradiente se multiplica repetidamente por valores menores a 1, volviéndose cada vez más pequeño y haciendo que las capas iniciales prácticamente no aprendan.

Actividad 3

¿Qué diferencias observa en la velocidad de convergencia entre ambos modelos?

Al comparar ambos modelos, se nota que ResNet18 despegue más rápido desde el principio. Ya en la primera época alcanzó un 20.63 % de accuracy en validación, mientras que la PlainNet18 se quedó con un 14.77 %. Esta diferencia se mantuvo durante todo el proceso; por ejemplo, al llegar a la época 10, aunque los valores de accuracy estaban cerca (53.32 % vs 55.04 %), la ResNet presentaba una pérdida de entrenamiento más baja (1.38 frente a 1.56). Esto demuestra que la arquitectura con conexiones residuales es mucho más eficiente para aprender. Por otro lado, la PlainNet18 tuvo un progreso mucho más lento y gradual, lo que confirma que, sin los atajos, a la red le cuesta más tiempo encontrar soluciones óptimas durante el entrenamiento.

¿Cómo afecta la eliminación de las conexiones residuales al desempeño final?

Los resultados muestran lo siguiente:

Modelo	Mejor val acc	Parámetros
ResNet18	72.27 %	11,220,132
PlainNet18	71.85 %	11,046,308

La diferencia final es de aproximadamente 0.4 puntos porcentuales. Aunque no sea tan grande, es importante notar que PlainNet18 tiene ~174K parámetros menos y aún así no logra igualar a ResNet18. Para esta profundidad (18 capas) y este dataset, las skip connections contribuyen principalmente a acelerar la convergencia y mejorar la estabilidad del entrenamiento.

¿Sus resultados experimentales son consistentes con las motivaciones teóricas de introducir conexiones residuales en ResNet?

Parcialmente sí. La motivación principal de ResNet es resolver el problema de degradación en redes profundas. La consistencia se observa principalmente en la convergencia más rápida (en especial en las primeras épocas) y en su mayor estabilidad. Sin embargo, la diferencia en accuracy final es pequeña (~0.4 %), lo que se puede explicar porque 18 capas no es suficientemente profundo para que el desvanecimiento de gradiente sea muy grande.

Sección 2: Detección de Objetos con YOLO

Actividad 4

¿En qué se diferencia la detección de objetos de la segmentación de imágenes?

La detección de objetos identifica y localiza objetos en una imagen mediante cajas delimitadoras rectangulares, entregando la clase y posición de cada objeto como un rectángulo. La segmentación asigna una etiqueta a cada píxel de la imagen, generando una máscara por cada objeto detectado. Esto permite conocer exactamente la forma del objeto, no solo su ubicación aproximada. La detección es más rápida y computacionalmente más barata; la segmentación es más costosa pero más precisa en cuanto a los contornos.

¿Cómo se adapta el enfoque de YOLO para generar máscaras de segmentación además de cajas delimitadoras?

YOLO añade una rama secundaria que funciona en paralelo a la detección de cajas. En este esquema, el modelo genera globalmente un conjunto de máscaras que capturan formas y estructuras base de toda la imagen, mientras que, para cada objeto detectado en la cuadrícula, predice un set de coeficientes específicos. La máscara final de cada objeto se obtiene mediante una combinación lineal, donde estos coeficientes actúan como instrucciones para mezclar las máscaras de manera que se ajusten a la silueta particular del objeto. Finalmente, la máscara resultante se recorta siguiendo las coordenadas del bounding box.

¿Cuáles son los principales desafíos de la segmentación en imágenes reales?

Los principales desafíos son: los objetos pequeños; que ocupan pocos píxeles y generan máscaras imprecisas, la oclusión; que impide ver partes del objeto y obligan al modelo a inferir contornos, los bordes difusos; que no tienen separación clara, los cambios de iluminación; que confunden regiones del fondo con partes del objeto y objetos muy cercanos; que pueden fusionarse en una sola máscara.

Actividad 5

¿Qué tan precisas son las máscaras generadas en distintos escenarios?

En la imagen urbana del semáforo, el modelo detectó 21 objetos con confianzas altas (autos hasta 91.1 %, semáforos entre 39–75 %, personas entre 35–85 %), y las máscaras de los autos bien visibles y separados son en general precisas. En el Parque Forestal, con 12 objetos detectados (personas y bicicletas entre 30–80 %), las máscaras de personas son razonables aunque con menor precisión en extremidades. En el Metro Línea 5, el modelo detectó correctamente el tren (87.2 %) y las personas de la primera línea, pero con confianzas bajas (30–57 %), lo que se refleja en máscaras menos detalladas.

¿Cómo se comporta el modelo en presencia de múltiples objetos o escenas complejas?

El modelo funciona razonablemente bien cuando los objetos están claramente separados. Sin embargo, ante escenas complejas muestra limitaciones claras. En la imagen del semáforo, no detecta un auto que está parcialmente cubierto por otro auto, en la imagen del Parque Forestal, detecta a una persona montando bicicleta pero no reconoce la bicicleta que tiene debajo, probablemente porque la bici esta muy de frente en la imagen y en el metro, solo detecta a las personas de la primera fila, ignorando a quienes están detrás o más al fondo.

¿En qué casos el modelo falla y por qué cree que ocurre esto?

Se identificaron varios casos de falla concretos. Primero, oclusión entre objetos: el auto tapado por otro auto en la imagen del semáforo no es detectado porque el modelo no tiene suficiente información visual del objeto ocluido para reconocerlo con confianza. Segundo, objetos pequeños o lejanos: el carabinero en el centro de la calle y el ciclista al fondo de la imagen del semáforo no son detectados, lo que se explica porque estos objetos ocupan muy pocos píxeles. Tercero, oclusión parcial por otro objeto de distinta clase: la bicicleta debajo de la persona en el parque no se detecta. Cuarto, en la imagen del metro, las personas más alejadas o detrás de otras no son detectadas, ya que el modelo prioriza los cuerpos completos. Finalmente, el semáforo peatonal es detectado como un semáforo independiente, lo cual es discutible ya que esto depende de con que tipo de semaforo fue entrenado el modelo (podría ser que se entreno con semaforos que no tienen el semaforo peatonal).

Actividad 6

¿Qué hiperparámetro tuvo mayor impacto en el desempeño?

El hiperparámetro con mayor impacto fue `imgsz=640`, que aumentó el mAP@50 de 0.2742 (baseline) a 0.3673, un incremento de $\sim 34\%$. Los resultados fueron:

Experimento	mAP@50 (Mask)	mAP@50-95 (Mask)
Baseline (lr=0.01, batch=16, imgsz=320)	0.2742	0.1580
Exp1: lr=0.0001	0.2742	0.1580
Exp2: batch=32	0.2690	0.1635
Exp3: imgsz=640	0.3673	0.2284
Exp4: epochs=60	0.2992	0.1688

El impacto de la resolución se explica por la presencia de detalles finos que requieren mayor resolución para ser segmentados con precisión. Cabe notar que el learning rate no tuvo impacto (su resultado es igual al Baseline).

¿En qué tipo de imágenes el modelo mejora o empeora?

Con `imgsz=640`, el modelo mejora claramente en objetos grandes y con forma distinguible: persona (mAP50 Mask: 0.835), canoa (0.694) y paddle (0.651). En cambio, falla o directamente no aprende en clases con pocas instancias o formas difusas: dog (0.0), mountain (0.0), trunk (0.191) y dirt (0.069). Las clases con muy pocas muestras en validación (dog=5 instancias, mountain=2) no permiten evaluación confiable.

¿Qué limitaciones persisten después del fine-tuning?

En primer lugar, el tamaño del dataset es importante; trabajar con solo 171 imágenes para entrenamiento y 49 para validación restringe mucho la capacidad de generalización y hace que las métricas no sean tan estables. Además hay desbalance en las clases (como se menciona en la pregunta anterior). Por otro lado, la naturaleza de ciertas clases como dirt o rock resultan difíciles ya que sus contornos son difusos y difíciles de delimitar con precisión. Además es importante notar que estamos usando la version nano.

Sección 3: Representaciones Visuales (DINOv2)

Actividad 7

¿Qué diferencias principales captura el CLS token en comparación con los patch tokens?

Según los resultados del notebook, DINOv2 genera un CLS token de dimensión (384,) y 256 patch tokens de dimensión (256, 384). El CLS token es un vector global que resume toda la imagen en una sola representación: captura qué hay en la imagen, las relaciones entre objetos y el contexto general. Los patch tokens, en cambio, son representaciones locales donde cada fila corresponde a una región de 14×14 píxeles de la imagen, capturando texturas, bordes y estructuras espaciales. El mapa de normas de los patch tokens visualizado en el notebook muestra que las activaciones más altas corresponden a las zonas más informativas de la imagen.

¿Qué tipo de representación considera más adecuada para tareas de clasificación?

Para clasificación, el CLS token es la representación más adecuada. Al ser un vector global que condensa la información de toda la imagen, contiene la información necesaria para asignar una clase. Además, es eficiente ya un solo vector es suficiente como entrada. Esto se puede ver en los resultados de la Actividad 9, ya que usando el CLS token de DINOv2 como entrada a un MLP se alcanza 74.2 % de accuracy en CIFAR-10 con solo 30 épocas.

¿Qué tipo de representación considera más adecuada para tareas de representación multimodal?

Para tareas multimodales los patch tokens son más adecuados. Permiten establecer correspondencias entre regiones específicas de la imagen y partes del texto. El CLS token, al comprimir todo en un vector, pierde la información espacial para generar descripciones detalladas.

Actividad 8

¿Cómo afecta el escalamiento de una imagen a la calidad del feature matching utilizando DINOv2?

Se realizó feature matching con DINOv2 sobre 10 pares de imágenes de objetos reales, donde cada par consiste en dos vistas distintas del mismo objeto con diferencias de perspectiva, escala o condiciones visuales. Los resultados obtenidos son:

Par	Objeto	Avg Score	Matches >0.6
1	Cerveza	0.885	164
2	Costanera	0.872	238
3	Génesis	0.908	252
4	Guitarra	0.926	144
5	Moto	0.915	244
6	Paine	0.925	237
7	Torre Eiffel	0.924	140
8	Volcán	0.936	248
9	Castillo	0.944	256
10	Park	0.947	253

Los scores son notablemente altos en todos los pares, variando entre 0.885 (Cerveza) y 0.947 (Park), con una media de aproximadamente 0.918. Todos los pares superan el umbral de similitud de 0.6, con entre 140 y 256 matches fuertes por par.

Se observa que los objetos con mayor variación de escala o encuadre entre vistas (como Guitarra con 144 matches y Torre Eiffel con 140) tienen menos matches fuertes (aunque mantienen scores altos). Esto sugiere que cuando la escala cambia, algunos patches dejan de encontrar correspondencias directas, reduciendo el número de matches (no así el score).

¿En qué medida los descriptores de DINOv2 son invariantes al cambio de escala?

Los resultados muestran que DINOv2 exhibe una invarianza muy robusta al cambio de escala. Todos los pares obtienen scores superiores a 0.88, lo que indica que las representaciones de patch tokens se mantienen similares.

La diferencia más notable entre pares no está tanto en el score sino en la *cantidad* de matches fuertes: los pares con objetos de textura uniforme o fondo complejo (Torre Eiffel con 140, Guitarra con 144) producen menos correspondencias que objetos con mas texturas en toda la imagen (la foto de Cerro Castillo con 256, o la foto del Parc Guell con 253).

Actividad 9

¿Cómo se compara el desempeño del MLP usando píxeles crudos versus features de DINOv2?

Los resultados del notebook son los siguientes:

Modelo	Test Acc final	Convergencia
MLP + Píxeles crudos	56.0 %	Lenta, mejora gradual
MLP + Features DINOv2	74.2 %	Rápida, 67.6 % en época 1

DINOv2 supera a los píxeles en más de 18 %. La diferencia en velocidad de convergencia es también evidente donde desde la primera época, DINOv2 alcanzó 67.6 % de accuracy en test (vs 46.0 % con píxeles), y se estabiliza alrededor de la época 6–8. El MLP sobre píxeles “crudos” mejora lentamente durante todas las 30 épocas, llegando a 69.1 % en entrenamiento vs 56.0 % en test.

¿Por qué las features preentrenadas con DINOv2 pueden mejorar el rendimiento respecto al uso directo de píxeles?

Al utilizar píxeles crudos, el MLP se ve obligado a aprender cosas básicas como detectar bordes y texturas antes de poder clasificar el objeto. En cambio, las features de DINOv2 ya pasaron por un proceso de preentrenamiento masivo en un Vision Transformer, donde el modelo aprendió a capturar la estructura de millones de imágenes de forma autosupervisada. Al extraer el CLS token de 384 dimensiones, ya contamos con una buena representación con mucho contenido. Esto simplifica el trabajo del MLP ya que solo debe aprender a separar las clases en este espacio de características, en vez de intentar interpretar la imagen desde cero.

¿Qué problemas o limitaciones puede ocasionar el uso de imágenes de 32×32 píxeles con DINOv2?

DINOv2 fue preentrenado con imágenes de 224×224 y opera con patches de 14×14 píxeles. Para usarlo con CIFAR-10 (32×32), hubo que hacer un resize a 224×224 . Esta amplificación introduce efectos de interpolación y genera imágenes que lucen muy distintas a las imágenes naturales de alta resolución con las que DINOv2 fue preentrenado.